

Extracting Spin-Orbit Coupling (SOC) Hamiltonians from FHI-aims for use in DeepH.

Abstract

This manual documents the workflow developed to extract noncollinear Spin-Orbit Coupling (SOC) Hamiltonians from FHI-aims localized atomic orbital outputs and process them into the block-sparse format utilized by the DeepH machine learning ecosystem. It covers the necessary modifications to the Fortran source code of FHI-aims to expose the spatial SOC $\mathbf{II}$ matrices, the mathematical reconstruction and Hermiticity constraints of the full $2N \times 2N$ complex Hamiltonian, and the automated Python parsing strategies, including both a modified DeepH extraction tool and a standalone scripting route, used to validate the workflow.

Table of Contents

1. FHI-aims SOC Extraction and DeepH Workflow Development (Pg 2)
 - Making a modified FHI-aims build
 - Simulation to reproduce MoS2 results
 - Directory Layout
2. Full workflow summary (Pg 5)
 - Modification of the FHI-aims Fortran Source Code
 - Automating the Test Case
 - Dumping the Real-Space Hamiltonians
 - DeepH Based Processing
3. Extraction of the Real-Space SOC Hamiltonian ($\mathbf{II}$ Matrix) (Pg 8)
 - Extraction Location
 - Format of `soc_matrix`
 - Implementation of the Matrix Dump
 - Reading the Matrix in Python
 - Integration and Hamiltonian Construction
4. DeepH Hamiltonian Processing Overview (Pg 13)

- How DeepH Processes Hamiltonians
- Change Log: `aims_soc_to_deeph.py` vs `aims_to_deeph.py`

5. Methodology: Two Routes to DeepH Sparse HDF5 (Pg 16)

- Route 1: "The DeepH Way"
- Route 2: "The Alternative Way"

FHI-aims SOC Extraction and DeepH Workflow Development

This instruction file will cover:

- Building a modified FHI-aims with SOC Hamiltonian file dumping.
- How to run a simulation workflow to reproduce MoS2 results.
- The Directory Layout.

Making a modified FHI-aims build with SOC Hamiltonian dumping.

Go to `.../FHIaims_SOC_extract_dev/modified_source_files` and you will see a modified version of `calculate_second_variational_soc.f90` and a `ReadMe.txt` showing the relevant code block and how to modify the source code.

- Make a copy of you current FHI-aims directory for development, and substitute this file into `src/soc`.
- Use `cmake` and `make` to form a new FHI-aims build with the modified source code.

Simulation to reproduce MoS2 results.

- Create a relevant environment Environment dependences: ASE, pyfhiaims, deephdock (python 3.12 or higher), pytest (for unit tests in another file checking hermicity)

```
conda create -n fhi-soc-dock
conda activate fhi-soc-dock
conda install -c conda-forge python=3.14.6
conda install -c conda-forge ase
conda install -c conda-forge pyfhiaims
pip install deepx-dock
```

- Go to `.../FHIaims_SOC_extract_dev/2D_materials_data/MoS2_reproduce/run_workflow_unitcell.py` and update the number of cores and relevant paths and directories at the beginning of main. Run this workflow, it may take some time depending on resources (only about 2 mins for my 32 core AMD-thread ripper)

```
python 2D_materials_data/MoS2_reproduce/run_workflow_unitcell.py
```

The soc_intermediate is a single-point scf calculation of the relaxed structures. Here we can see that the DUMP SOC MATRIX block was activated and we now have two extra files "realspace_soc_matrix.out" and "realspace_soc_unfmttd.out". The Human readable realspace matrix is ~ 4x the size as rs_hamiltonian.out. However rs_hamiltonian.out has 1 column while realspace_soc_matrix.out has 3 columns of the 3 different Pi matrices.

- For band structure plotting, I installed and used climis

```
clims-aimsplot --band --species_dos --total_dos --emin -5 --emax 5 --save_only
clims-aimsplot --band --emin -5 --emax 5 --save_only
```

- Go to `.../FHIaims_SOC_extract_dev/DeepH_soc_process/run_aims_soc_to_deeph.py`, change base dir and run. uncomment `base_dir = current_dir.parent / '2D_materials_data/MoS2_reproduce'` to change base dir

run using:

```
python DeepH_soc_process/run_aims_soc_to_deepH.py
```

- This will create a new directory called `.../`

```
FHIaims_SOC_extract_dev/2D_materials_data/MoS2_reproduce/deepH_files
```

Directory Layout

The workspace is organized into several key subdirectories:

1. `md_files/` (Documentation)

Contains in-depth markdown documentation explaining the physics, methodology, and modifications made throughout this project.

- **modified_deepH_code_for_SOC.md**: Details modifications to DeepH's parser for SOC.
- **compare_parsers_methodology.md**: Explains how the DeepH-based parsing route was mathematically verified against an explicit custom Python parser.
- **extract_realspace_soc.md**: A guide on how the real-space SOC Hamiltonian ($\mathbf{H}$ matrix) is identified, dumped from FHI-aims, and processed in Python.
- **full_workflow_summary.md**: A comprehensive overview of the full pipeline (from Fortran source code to HDF5 tensors).

2. `modified_source_files/` (FHI-aims Source Modifications)

Contains the specific Fortran files modified within FHI-aims to support SOC extraction.

- `calculate_second_variational_soc.f90`: The patched source file that natively dumps the three spatial components of the SOC operator (π_x, π_y, π_z) to `realspace_soc_matrix.out` alongside standard scalar Hamiltonians.
- `ReadMe.txt`: Notes pertaining to the FHI-aims source patches.

3. `DeepH_soc_process/` (DeepH Conversion Scripts)

Contains the production-ready python parsers designed to ingest FHI-aims outputs and convert them into the block-sparse format utilized by DeepH.

- `aims_soc_to_deepH.py`: The modified DeepH extraction tool capable of handling the

noncollinear $2N \times 2N$ complex SOC Hamiltonian.

- `run_aims_soc_to_deep.py` : An execution wrapper to run the extraction locally.

4. `script_testing/` (Validation and Unit Tests)

A dedicated testing suite verifying that matrix parsing and mathematical reconstruction (including Hermiticity and Parity constraints) are perfectly maintained.

- `compare_parsers.py` : Compares the output of `aims_soc_to_deep.py` against a custom mathematical script (`build_soc_hamiltonian.py`) to ensure 100% numerical consistency.
- `unit_tests/` : Pytest suite to rigorously validate the parser logic and matrix parity constraints (`symm_signs`).

5. `2D_materials_data/` (Automated Test Workflows)

End-to-end Python automation scripts (`run_workflow_unitcell.py`) and associated data evaluating SOC properties on canonical 2D materials. These scripts:

- Perform structural relaxations (e.g., `relaxation_light`).
- Evaluate Spin-Orbit Coupling single points (e.g., `soc_intermediate`).
- Trigger the custom Fortran dump to produce SOC matrices for the DeepH parser.

See the two directories for graphene and MoS2. I used `clims` to plot the band structures and obviously MoS2 has non-negligible SOC effects. These tests are only a unit cell so it runs quite quickly.

Summary

This package provides everything needed to reproduce, validate, and utilize the FHI-aims to DeepH SOC data pipeline. For an in-depth understanding of the mathematical operations involved, refer to the documentation inside `md_files/` .

Full workflow summary.

This document details the pipeline developed to extract noncollinear Spin-Orbit Coupling (SOC) Hamiltonians from FHI-aims and seamlessly process them into the DeepH machine learning ecosystem.

1. Modification of the FHI-aims Fortran Source Code

In standard periodic Density Functional Theory (DFT) calculations using FHI-aims, Spin-Orbit Coupling is typically applied perturbatively or self-consistently onto the localized atomic orbital (LAO) basis. To utilize these matrices for machine learning, we needed raw access to the underlying spatial real-space components of the SOC operator before the generalized eigenvalue solver diagonalizes the system. Consequently, we modified the FHI-aims Fortran source code to actively dump these operator matrices during the real-space integration phase.

Specifically, alongside the standard real-space scalar Hamiltonian (H_{scalar}) written to `rs_hamiltonian.out`, the modified code now dumps the three spatial components of the SOC operator:

- π_x (the x -component)
- π_y (the y -component)
- π_z (the z -component)

which are written to a new output file called `realspace_soc_matrix.out`.

2. Automating the Test Case (`run_workflow_unitcell.py`)

Note, you must edit the paths and directories in `run_workflow_unitcell.py` to suit your system.

To validate the modified Fortran code, a Python automation script (`2D_materials_data/MoS2_unitcell/run_workflow_unitcell.py`) orchestrates the end-to-end DFT calculation.

1. **Structure Generation:** It utilizes ASE (Atomic Simulation Environment) and PyFHIaims to generate a 2D layer object of Molybdenum Disulfide (MoS2).
2. **Slab Relaxation:** It executes a geometric structural relaxation using `light` species defaults and Van der Waals corrections to better capture inter-layer interactions.
3. **Spin-Orbit Single Point Evaluation:** It then initiates a follow-up single-point calculation using `tight` species defaults with the Spin-Orbit Coupling (`include_spin_orbit`) keyword explicitly enabled.
4. **Data Extraction:** During this final noncollinear subroutine, the hardcoded Fortran block we introduced is actively triggered, prompting FHI-aims to physically dump the relevant

SOC and scalar matrices to the working directory.

3. Dumping the Real-Space Hamiltonians

Once a calculation concludes, the FHI-aims working directory contains the necessary raw outputs:

- `rs_indices.out` : Defines the sparse non-zero elements, mapping them to specific basis functions (orbitals) and spatial displacement vectors ($\mathbf{R}$).
- `rs_hamiltonian.out` : Contains the $N \times N$ scalar Hamiltonian (H_{scalar}) evaluating the core electrostatic, kinetic, and non-relativistic interaction terms.
- `realspace_soc_matrix.out` : Contains the decoupled π_x, π_y, π_z matrices corresponding to the SOC angular momentum operators, 3 columns of x, y, z pauli pi matrices.
- `rs_overlap.out` : Contains the $N \times N$ spatial overlap matrix (S).

Because these are evaluated on a strictly localized, non-orthogonal atomic orbital basis, the matrices reflect the raw generalized eigenvalue problem $HC = SCE$.

4. DeepH Based Processing (`aims_soc_to_deeph.py`)

DeepH requires the Hamiltonian to be formatted as a complex, $2N \times 2N$ block-sparse tensor stored in chunked `.h5` files. To make this compatible we copied and adapted the DeepH data translator, `aims_to_deeph.py`, to form `aims_soc_to_deeph.py`, which now acts as a standalone parser for FHI-aims SOC Hamiltonians.

Mathematical Reconstruction of the SOC Tensor

The parser reads the four scalar fields ($H_{scalar}, \pi_x, \pi_y, \pi_z$) mapping them across the sparsity indices defined in `rs_indices.out`.

It then combines them into the full Noncollinear $2N \times 2N$ complex block matrix using the standard Pauli matrix formulations. For any orbital pair (i, j) at displacement $\mathbf{R}$, the 2×2 spin-subblock is constructed as:

$$H_{i,j}(\mathbf{R}) = \begin{pmatrix} H_{\uparrow\uparrow} & H_{\uparrow\downarrow} \\ H_{\downarrow\uparrow} & H_{\downarrow\downarrow} \end{pmatrix}$$

Where the individual tensor elements are reconstructed as: $H_{\uparrow\uparrow} = H_{scalar} - i\pi_z$ $H_{\downarrow\downarrow} = H_{scalar} + i\pi_z$ $H_{\uparrow\downarrow} = -i\pi_x - \pi_y$ $H_{\downarrow\uparrow} = -i\pi_x + \pi_y$

Resolving Spatial Hermiticity

A major challenge arose regarding DeepH's native spatial Hermiticity assumptions:

$$H(\mathbf{R}) = H(-\mathbf{R})^\dagger.$$

When equating the real and imaginary parts of the 2×2 block tensor, it dictates strict physical rules for the raw outputs: the scalar Hamiltonian must be geometrically symmetric, while the spatial SOC Pauli matrices (π_x, π_y, π_z) must be geometrically anti-symmetric.

We resolved Hermiticity conflicts by configuring the parser with `symm_signs=[1, 1, -1, -1, -1]`, forcing DeepH to accurately interpret the spatial SOC matrices with antisymmetric parity. Additionally, because the SOC matrices are integrated numerically over a real-space grid in FHI-aims, discrete grid artifacts lead to some noise ($\sim 10^{-6}$). By including an adjustable DeepH Hermiticity `tolerance` argument (default `1e-3`), the pipeline can account for the material-dependent grid noise and validate the parsed data.

Dual output of SOC and non-SOC Hamiltonians

Finally, to support diverse model training objectives, the parser was configured to simultaneously output two formats:

1. `soc-hamiltonian.h5`: The full $2N \times 2N$ complex SOC Hamiltonian representing the fully reconstructed equations above.
2. `hamiltonian.h5`: The purely non-SOC baseline, generated by feeding H_{scalar} directly into the diagonal spin blocks as a real matrix. Note, these are named `hamiltonian.h5` and `hamiltonian0.h5` in the original DeepH data translator.

Extraction of the Real-Space SOC Hamiltonian (Π Matrix)

The real-space SOC Hamiltonian is calculated before being transformed into the state-space basis. This matrix is referred to as the Π matrix in the literature, and in the FHI-aims source code, it is represented by the variable `soc_matrix`. This document details how this matrix was extracted and dumped to a file for later post-processing (e.g., using Python) as part of our ML Hamiltonian workflow.

1. Extraction Location

The Π matrix is allocated and constructed in the main SOC driver routine in the FHI-aims codebase: **File:** `soc_source/calculate_second_variational_soc.f90`

The extraction logic was added directly after the following subroutine call (around line 664) where the `soc_matrix` is populated:

```
! Fill in the matrix for the real-space SOC matrix elements (\Pi from the
! Phys Rev Mater. paper)
call get_timestamps(time_matrix, clock_time_matrix)
! Step 3 in Section III.3 from Huhn and Blum, Phys. Rev. Mater. (2017)

call integrate_soc_matrix (rho,hartree_potential,partition_tab,soc_matrix)
*** ADDED HERE ***
```

Directly after this call, the `soc_matrix` variable contains the compressed real-space SOC Hamiltonian. The dump was placed here before any subsequent rotation (e.g., by `rotate_Pi_columns` if `calculate_mae` is true).

2. Format of `soc_matrix`

`soc_matrix` is a 2D array allocated as: `real*8, dimension(ld_soc_matrix, 3) :: soc_matrix`

- **First dimension (`ld_soc_matrix`):** Represents the packed pairs of basis functions. Because the Hamiltonian matrix is highly sparse in real-space, FHI-aims packs the non-zero matrix elements into a contiguous 1D array per operator. `ld_soc_matrix` is either `n_hamiltonian_matrix_size` (full size) or `batch_perm(n_bp_integ)%n_local_matrix_size` (if load balancing is used).
- **Second dimension (`3`):** Represents the three spatial coordinates (x, y, z) of the spin-orbit coupling operator.

3. Implementation of the Matrix Dump

To dump this matrix, an unformatted (binary) write statement was injected to handle the large matrix efficiently.

The write statement was restricted to a single MPI task (`myid == 0`), assuming runs

without `use_local_index` (where every process has the full array). The following code block was added to the FHI-aims source right after `call integrate_soc_matrix(...)`:

```
! --- DUMP SOC MATRIX ---
! --- MODIFICATION BY Adam Coxson
if (myid == 0) then
  ! Unformatted dump (smaller file, faster read in Python)
  open(unit=999, file="realspace_soc_matrix_unfmttd.out", form='unformatted',
status='replace')
  ! Write the dimensions first so the reader script knows the size
  write(999) ld_soc_matrix
  write(999) 3
  ! Write the actual matrix block
  write(999) soc_matrix(1:ld_soc_matrix, 1:3)
  close(999)

  ! Alternative: Formatted dump (easier to debug, but larger file)
  open(unit=998, file="realspace_soc_matrix.out", status='replace')
  write(998, *) ld_soc_matrix, 3
  do i = 1, ld_soc_matrix
    write(998, '(3E24.15)') soc_matrix(i, 1:3)
  end do
  close(998)
end if
! -----
```

4. Reading the Matrix in Python

The unformatted (binary) dump is loaded into Python using `scipy.io.FortranFile` during post-processing. Because it is dumped as a compressed vector of overlapping basis pairs, the sparsity pattern array is required to map it back to basis pairs. The formatted dump is also performed and this file is utilised in the down stream tasks in the example workflows.

The following Python snippet demonstrates how the binary data is read in to python:

```
import numpy as np
from scipy.io import FortranFile

# Read the dumped binary file
f = FortranFile('realspace_soc_matrix_unfmttd.out', 'r')

# FHI-aims wrote unformatted records. The dimensions are read first:
ld_soc_matrix = f.read_ints(dtype=np.int32)[0] # read_ints() advances pointer to
# next Fortran record
num_coords = f.read_ints(dtype=np.int32)[0]

# Then the actual array is read:
soc_matrix_packed = f.read_reals(dtype=np.float64)
# Reshape taking Fortran column-major order into account
soc_matrix_packed = soc_matrix_packed.reshape((num_coords, ld_soc_matrix)).T
f.close()

print(f"Loaded packed SOC matrix of shape: {soc_matrix_packed.shape}")
```

Alternatively, the formatted dump (`realspace_soc_matrix.out`) can be read directly into Python using `numpy` :

```
import numpy as np

# Read the SOC matrix, skipping the first line (dimensions)
soc_data = np.loadtxt('realspace_soc_matrix.out', skiprows=1)
Pi_x = soc_data[:, 0]
Pi_y = soc_data[:, 1]
Pi_z = soc_data[:, 2]

print(f"Loaded formatted SOC matrix of shape: {soc_data.shape}")
```

5. Integration and Hamiltonian Construction

The parsing of the dumped `soc_matrix` can be performed using two methods within this workflow: via a modified **DeepH Dock** or using a standalone Python script (`build_soc_hamiltonian.py`).

Method A: DeepH Dock Integration

The parsing of the dumped `soc_matrix` into **DeepH Dock** utilises the following formats:

- **Sparsity Pattern:** The `soc_matrix` shares the exact same sparsity pattern and indexing format as the scalar-relativistic Hamiltonian.
- **Index File:** DeepH Dock natively parses `rs_indices.out` to map the 1D compressed vector to the basis pairs. Because the indexing is identical, the standard `rs_indices.out` generated by FHI-aims (via `output_rs_matrices plain/hdf5`) was used to unpack the `soc_matrix_packed` array.
- **Channel Count:** The extracted `soc_matrix` contains **3 channels** representing the x, y, z spatial components of the SOC operator $\vec{\Pi}$.

To integrate this with the DeepH conversion scripts, the 3-channel numpy array (`soc_matrix_packed`) was loaded, and the same CSR/COO matrix construction routine used for `rs_hamiltonian.out` was applied, using the indices directly from `rs_indices.out`.

This relies on modifying and overwriting functions from DeepH, which can be found in `aims_soc_to_deeph.py`.

Method B: Standalone Construction Script

Alternatively, the formatted output can be processed using the standalone script `script_testing/build_soc_hamiltonian.py`. This script constructs the full Spin-Orbit Coupled Hamiltonian in the Localized Atomic Orbital (LAO) basis directly from the real-space output matrices.

The script operates by:

1. Parsing the `rs_indices.out` file to retrieve the sparsity pattern and mapping logic.
2. Loading the scalar-relativistic Hamiltonian (`rs_hamiltonian.out`) and the formatted SOC matrix (`realspace_soc_matrix.out`).
3. Constructing the final complex spin-dependent Hamiltonian matrix in a spin-blocked format by mapping the spatial operators and verifying its Hermiticity.

You can run this script directly to yield a finalized `H_soc_rs.npy` Hamiltonian for further analysis:

```
python script_testing/build_soc_hamiltonian.py \
  --dir path/to/aims/output \
  --indices rs_indices.out \
  --hamiltonian rs_hamiltonian.out \
  --soc realspace_soc_matrix.out \
  --output H_soc_rs.npy
```

Validation: Unit testing using `pytest` has confirmed that when the 3-channel matrix is packed alongside the scalar field with properly mapped anti-symmetric parity (`symm_signs=[1, 1, -1, -1, -1]`), the resulting complex $2N \times 2N$ Hamiltonian satisfies both Spatial Hermiticity ($H(\mathbf{R}) = H(-\mathbf{R})^\dagger$) and Time-Reversal Symmetry. The finalized outputs are dumped as `soc-hamiltonian.h5` and `hamiltonian.h5` for immediate use.

DeepH Hamiltonian Processing Overview

This document provides a high-level overview of how DeepH processes Hamiltonians from FHI-aims outputs, and details the specific modifications made to the original parser to support Noncollinear Spin-Orbit Coupling (SOC).

How DeepH Processes Hamiltonians

DeepH uses a data processing pipeline to map real-space data from density functional theory (DFT) codes into its sparse HDF5 format. The workflow for parsing Hamiltonians follows these general steps:

- 1. High-Level Abstraction:** The entry point is a translator class (e.g., `PeriodicAimsDataTranslator`) which controls the data conversion via a master function (e.g., `transfer_one_aims_to_deeph`).
- 2. Raw Matrix Ingestion:** FHI-aims outputs scalar matrices (H_{scalar}), and in the SOC case, the Pauli matrices (π_x, π_y, π_z) as raw 1D array lists.
- 3. Assembly and Mapping:** DeepH relies on mapping utilities (such as `_trans_mxs_to_R_dict` and `_trans_R_dict_to_entries`). These functions loop over the FHI-aims sparsity index file (`rs_indices.out`), apply spherical harmonic parity factors, perform unit conversions (`HARTREE_TO_EV`), and assert spatial Hermiticity constraints.

4. **Tensor Recombination:** The reconstructed blocks are passed to concatenation routines to build the final block-sparse tensors. For collinear spin, they are placed diagonally. For noncollinear SOC, they are combined into complex 2×2 spin-orbital sub-blocks.
5. **Data Export:** The final tensors, alongside auxiliary structures like `info.json`, `POSCAR`, and overlaps, are written to sparse `.h5` formats suitable for machine learning workflows.

Our customized noncollinear SOC parser (`aims_soc_to_deeph.py`) builds upon this pipeline by introducing explicit Pauli tensor algebra while preserving the internal mapping logic of DeepH.

Change Log: `aims_soc_to_deeph.py` VS `aims_to_deeph.py`

The following modifications were made to the original DeepH dock parser (`aims_to_deeph.py`) to create the SOC variant (`aims_soc_to_deeph.py`), ensuring the $2N \times 2N$ complex SOC Hamiltonian is extracted and reconstructed.

1. Control Parameter Parsing (`_read_ctrl_in`)

- **Original** (`aims_to_deeph.py`): Only detected collinear spin settings via `spin collinear`.
- **Modified** (`aims_soc_to_deeph.py`): Added detection for `include_spin_orbit` in `control.in`. When present, it correctly flags the data as SOC-enabled (`spin_orbit = True`).

2. Reading SOC Pauli Matrices (`_read_soc_matrices`)

- **Original:** Only read the scalar `rs_hamiltonian.out` (and its spin up/down counterparts).
- **Modified:** Introduced the `_read_soc_matrices` function to parse `realspace_soc_matrix.out`. FHI-aims dumps the three spatial π_x, π_y, π_z matrices inside this file. The script parses the array and returns the individual matrix components, correctly applying the unit conversion (`HARTREE_TO_EV`).

3. Data Structure Packing (`analysis_data`)

- **Original:** Packed only the `ovlp` and `H_scalar` matrices into the translation processing list (`self.mx_lst`).
- **Modified:** When `spin_orbit` is True, it packs `ovlp`, `H_scalar`, and all three Pauli

matrices (`pi_x`, `pi_y`, `pi_z`) into `self.mx_lst` .

4. Spin-Orbit Tensor Recombination (`_soc_concatenate`)

- **Original:** Only featured `_spin_concatenate` which placed H_{up} and H_{dn} on the diagonal of a block matrix for collinear calculations.
- **Modified:** Added the complex concatenation routine `_soc_concatenate` . After the spatial matrices are mapped by the DeepH parsing algorithms, this routine combines them into the full $2N \times 2N$ complex block-sparse array using the following mathematical formulation:

- $H_{\uparrow\uparrow} = H_{scalar} - i\pi_z$
- $H_{\downarrow\downarrow} = H_{scalar} + i\pi_z$
- $H_{\uparrow\downarrow} = -i\pi_x - \pi_y$
- $H_{\downarrow\uparrow} = -i\pi_x + \pi_y$

5. Ensuring Spatial Hermiticity (`symm_signs`)

- **Original:** DeepH executes a spatial Hermiticity check inside `_trans_mxs_to_R_dict` that mirrors matrices across $\mathbf{R}$ and $-\mathbf{R}$. For collinear scalar outputs, this works via symmetric pairing.
- **Modified:** To construct a valid complex $2N \times 2N$ Hamiltonian, the spatial components must obey explicit parity constraints: the scalar matrices are symmetric, but the SOC Pauli matrices (π_x, π_y, π_z) are anti-symmetric. We enforce this by setting `symm_signs = [1, 1, -1, -1, -1]` . Furthermore, because FHI-aims computes the SOC real-space integrations numerically on a grid, a rigid Hermiticity threshold can fail due to grid noise. We exposed an adjustable internal DeepH validation `tolerance` (default `1e-3`) to map the SOC matrices while maintaining Hermiticity validations and accounting for grid noise across different materials.

6. Dual Export Naming Convention

- **Original:** For collinear models, `export_H0` looked for separate files like `rs_hamiltonian0.out` to isolate the base Hamiltonian.
- **Modified:** To support various training workflows, the parsing pipeline generates two distinct files. It outputs the complex SOC tensor as `soc-hamiltonian.h5` and the purely scalar base Hamiltonian (mapped diagonally into a $2N \times 2N$ real block) as `hamiltonian.h5` . This allows extraction of both the SOC and non-SOC representations in a single run.

7. Package Portability Adjustments

- **Modified:** To allow `aims_soc_to_deeph.py` to act as an independent, locally executed wrapper script overriding the installed package, the `transfer_all_aims_to_deeph` function and the import for `parallel_map` were removed. This prevents conflicts with the installed `deepx_dock` environment, ensuring that the local file pulls all its other dependencies (like constants and math utilities) directly from the active Conda package.

Methodology: Two Routes to DeepH Sparse HDF5

The `compare_parsers.py` script validates our noncollinear spin-orbit coupling (SOC) extraction process by constructing the identical $2N \times 2N$ complex block-sparse Hamiltonian using two independent scripts: the modified DeepH Dock functions and standalone `build_soc_hamiltonian.py` script.

Despite their different architectures, both routes yield 100% identical numerical outputs, proving that our matrix formulations are accurate and compatible with the `deepx_dock` environment.

Route 1: "The DeepH Way" (`run_deeph_way`)

This route utilizes the native, object-oriented ecosystem of the DeepH data translation module.

How it works:

1. **High-Level Abstraction:** It instantiates the `PeriodicAimsDataTranslator` from `aims_soc_to_deeph.py` and triggers `transfer_one_aims_to_deeph`.
2. **Internal DeepH Mapping:** It relies heavily on deeply nested `deepx_dock` utilities (like `_trans_mxs_to_R_dict` and `_trans_R_dict_to_entries`).
3. **Array Packaging:** The FHI-aims outputs ($H_{\text{scalar}}, \pi_x, \pi_y, \pi_z$) are passed as a raw list of 1D arrays into the mapping utility.
4. **Automated Assembly:** The deep-level functions automatically loop over the FHI-aims sparsity indexing (`rs_indices.out`), apply spherical harmonic parity factors (`phase_factor`), apply the unit conversions (`HARTREE_TO_EV`), and historically execute

spatial Hermiticity assertions.

5. **Tensor Recombination:** The recombined blocks are eventually passed to `_soc_concatenate` and `_spin_concatenate` to build the complex tensors, which are then natively dumped to `.h5` files.

Pros:

- Deeply integrated into the standard machine-learning pipeline.
- Automatically handles auxiliary file creation (`info.json` , `POSCAR` , `overlap.h5`).

Route 2: "The Alternative Way" (`custom_alternative_way`)

This route bypasses DeepH's native Hamiltonian mapping ecosystem entirely, replacing it with explicit, transparent, custom Python loop logic to verify the mathematical formulations independently.

How it works:

1. **Hybrid Foundation:** It uses DeepH strictly to construct auxiliary non-Hamiltonian files (like `POSCAR` and `info.json`), then handles the Hamiltonian autonomously.
2. **Raw Matrix Ingestion:** It reads `rs_hamiltonian.out` and `realspace_soc_matrix.out` independently and applies the `HARTREE_TO_EV` scalar directly to the flat arrays.
3. **Explicit Spatial Block Construction:**
 - Instead of using DeepH's recursive abstractions, it executes a manual `for` loop over `n_cells` (**R**) and `n_basis` (orbitals).
 - It builds a dictionary `mx_R` where keys are the spatial vectors and atom pairs `(R_1, R_2, R_3, i_atom, j_atom)`.
 - The matrix elements are actively sorted and placed into multi-dimensional NumPy arrays representing specific atomic sub-blocks.
4. **Manual Pauli Tensor Application:**
 - Once the spatial blocks are assembled, it explicitly reconstructs the 2×2 spin-orbital sub-blocks mathematically:
 - $H_{\uparrow\uparrow} = H_{\text{scalar}} - 1j \cdot \pi_z$
 - $H_{\downarrow\downarrow} = H_{\text{scalar}} + 1j \cdot \pi_z$
 - $H_{\uparrow\downarrow} = -1j \cdot \pi_x - \pi_y$

$$\blacksquare H_{\downarrow\uparrow} = -1j \cdot \pi_x + \pi_y$$

5. **Sparse Packaging:** It manually computes the `chunk_boundaries`, `chunk_shapes`, and `atom_pairs` required by DeepH and uses the standard `h5py` library to dump the arrays to `soc-hamiltonian.h5` and `hamiltonian.h5`.

Pros:

- Mathematically transparent and highly readable.
 - Easy to debug and verify because the matrix tensor math ($H_{\uparrow\downarrow}$, etc.) is applied to fully assembled $N \times N$ block matrices rather than scattered 1D arrays.
 - Proves conclusively that the native DeepH route is interpreting the SOC data correctly.
-